

Review Finding App - Backend Documentation

Ajil V Sajith

September 12, 2025

1 Overview

The **Review Finding App** is a backend system built with **Django** and **Django REST Framework**. It allows users to submit text reviews (focused on clothing products) and automatically predicts a rating for the review text using a machine learning model (**DistilBERT**).

Key Features

- Users can submit clothing-related reviews.
- The backend validates review text to ensure quality.
- The text is passed to a machine learning service.
- The ML service predicts a **rating between 1–5**.
- Users can filter and sort reviews via query parameters.

2 Data Model

The system has a single model for storing reviews:

```
class Review(models.Model):
    name = models.CharField(max_length=100, blank=True, null=True)
    text = models.TextField()
    predicted_rating = models.FloatField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        display_name = self.name if self.name else "Anonymous"
        return f"{display_name}: {self.text[:30]}... ({self.
            predicted_rating})"
```

3 Machine Learning Integration

The backend uses a **fine-tuned DistilBERT model** to predict ratings.

- Model: DistilBERT (distilbert-base-uncased)
- Output: Class index mapped to a rating (1–5)
- Device: Runs on GPU if available, otherwise CPU
- Input: Review text (max length = 128 tokens)

```
def predict_rating(text: str) -> float:
    model, tokenizer, _ = load_model()
    inputs = tokenizer(text, return_tensors="pt",
                       truncation=True, padding=True,
                       max_length=128).to(_device)
    with torch.no_grad():
        outputs = model(**inputs)
        pred_class = torch.argmax(outputs.logits, dim=1).cpu().item()
    return float(pred_class + 1.0)
```

4 API Endpoints

The backend exposes a single endpoint for handling reviews.

1. List and Create Reviews

- URL: /reviews/
- Methods: GET, POST

Request - Create Review (POST)

```
POST /reviews/
{
  "name": "Alice",
  "text": "The fabric is very comfortable and stylish!"
}
```

Response - Create Review (201 Created)

```
{
  "id": 1,
  "name": "Alice",
  "text": "The fabric is very comfortable and stylish!",
  "predicted_rating": 5.0,
  "created_at": "2025-09-12T12:30:45Z",
  "updated_at": "2025-09-12T12:30:45Z"
}
```

Request - List Reviews (GET)

```
GET /reviews/  
GET /reviews/?sort_by=created  
GET /reviews/?rating=5  
GET /reviews/?rating_gte=3&rating_lte=5
```

Response - List Reviews

```
[  
  {  
    "id": 1,  
    "name": "Alice",  
    "text": "The fabric is very comfortable and stylish!",  
    "predicted_rating": 5.0,  
    "created_at": "2025-09-12T12:30:45Z",  
    "updated_at": "2025-09-12T12:30:45Z"  
  }  
]
```

5 Filtering and Sorting

The API supports query parameters for filtering:

- `?sort_by=created` - Sort by creation date
- `?sort_by=updated` - Sort by last updated
- `?rating=` - Filter by exact rating
- `?rating_gte=` - Filter by rating greater or equal
- `?rating_lte=` - Filter by rating less or equal

6 Conclusion

This backend provides a scalable and ML-driven system for managing and rating product reviews. It can be extended to support:

- Multiple product categories
- User authentication
- Advanced recommendation systems